
Práctica Guiada: Administrando Contenedores y Servicios Web con Docker y Docker Compose

Implantación de Aplicaciones Web

Índice

FASE 1: Primeros Pasos con Apache en Docker	2
📸 Capturas requeridas para la memoria:	2
FASE 2: Creación de Imagen Personalizada y Docker Hub	3
📸 Capturas requeridas para la memoria:	3
FASE 3: Laboratorio de Diagnóstico y Resolución de Errores	3
Escenario A: Conflicto de Puertos	4
Escenario B: Ruta Incorrecta en el Dockerfile	4
Escenario C: Caída Inmediata del Contenedor (Crash)	4
📸 Capturas requeridas para la memoria:	4
FASE 4: Orquestación con Docker Compose	4
📸 Capturas requeridas para la memoria:	5

Práctica: Docker

- Operar contenedores desde la CLI aplicando mapeo de puertos y gestión de procesos.
- Diseñar e interpretar un **Dockerfile** para construir una imagen personalizada basada en **Apache (httpd)**.
- Publicar e interactuar con imágenes en el registro público **Docker Hub**.
- Diagnosticar y resolver fallos comunes mediante el análisis de logs y estado de contenedores.
- Automatizar el despliegue de múltiples servicios utilizando **Docker Compose**.

FASE 1: Primeros Pasos con Apache en Docker

Reto: Debes poner en marcha un servidor web Apache ligero utilizando la versión **alpine** de la imagen oficial.

1. **Despliegue inicial:**
 - Busca en Docker Hub la imagen oficial de Apache (**httpd**) en su variante **alpine**.
 - Lanza un contenedor en segundo plano que redirija el puerto **8080 de tu equipo local** al puerto interno estándar donde escucha Apache (**80**).
 - Asigna un nombre identificativo a tu contenedor.
2. **Comprobación y Registros (Logs):**
 - Accede desde tu navegador web a la dirección local correspondiente y confirma que el servidor responde.
 - Visualiza los logs en tiempo real mediante la consola de Docker y genera peticiones en el navegador para verificar cómo se registran los accesos HTTP (códigos **200 OK**, etc.).



Capturas requeridas para la memoria:

- *Captura 1.1:* Salida de la consola con la ejecución del comando para arrancar el contenedor y listado del contenedor en ejecución (**ps**).
- *Captura 1.2:* Navegador web cargando la página por defecto junto a la terminal mostrando los registros del servidor (**logs**).

FASE 2: Creación de Imagen Personalizada y Docker Hub

Reto: Crearás un sitio web corporativo propio dentro del contenedor, empaquetarás el resultado como una imagen reutilizable y la publicarás en tu cuenta personal de Docker Hub.

1. Construcción de la Web y el `Dockerfile`:

- Crea un directorio de trabajo.
- Crea un archivo `index.html` con una estructura HTML básica que incluya tu nombre, fecha y un título.
- Escribe un archivo `Dockerfile` que utilice como base la imagen `httpd:alpine`, copie tu archivo `index.html` a la ruta interna donde Apache busca las páginas por defecto, y exponga el puerto web.



Pista para la ruta de Apache: En las imágenes oficiales basadas en Alpine, el directorio raíz del servidor web no suele ser `/var/www/html/`, sino `/usr/local/apache2/htdocs/`.

2. Publicación en Docker Hub:

- Regístrate/inicia sesión en [Docker Hub](#).
- Inicia sesión desde tu terminal.
- Construye tu imagen asignándole una etiqueta (*tag*) que incluya tu nombre de usuario de Docker Hub (ejemplo: `tu_usuario/mi-apache-web:v1.0`).
- Subela a tu repositorio remoto en Docker Hub.

3. Prueba de Descarga:

- Elimina la imagen local y el contenedor que creaste para hacer la prueba.
- Descarga (*pull*) tu imagen desde Docker Hub y vuelve a levantarla en un puerto distinto (ejemplo: **8081**).



Capturas requeridas para la memoria:

- *Captura 2.1:* Contenido del archivo `Dockerfile`.
- *Captura 2.2:* Proceso de subida (*push*) completado con éxito en la consola.
- *Captura 2.3:* Tu repositorio público visible en la web de Docker Hub.
- *Captura 2.4:* Ejecución del contenedor descargado desde Docker Hub funcionando en el navegador.

FASE 3: Laboratorio de Diagnóstico y Resolución de Errores

Reto: En esta fase te enfrentarás a problemas reales. Tendrás que provocar los fallos, analizar la información disponible y explicar cómo los has solucionado.

Escenario A: Conflicto de Puertos

1. Intenta ejecutar un nuevo contenedor asignándole en el equipo anfitrión el mismo puerto que ya está utilizando otro contenedor activo.
2. Lee el error devuelto por la terminal.
3. Explicación requerida: ¿Por qué ocurre y qué comando o parámetro debes modificar para solventarlo?

Escenario B: Ruta Incorrecta en el Dockerfile

1. Modifica tu `Dockerfile` introduciendo una ruta de destino intencionadamente errónea para el comando de copia del HTML (por ejemplo, `/var/www/html/`).
2. Construye y ejecuta la imagen.
3. Al acceder al navegador notarás que la web no muestra tu archivo personalizado.
4. Inspección requerida: Accede de forma interactiva a la consola interna del contenedor o inspecciona la ruta con comandos de Docker para verificar qué archivos hay en la ruta oficial de Apache.

Escenario C: Caída Inmediata del Contenedor (*Crash*)

1. Modifica la instrucción de arranque por defecto (`CMD` o `ENTRYPOINT`) de un `Dockerfile` escribiendo un parámetro o comando que no exista.
2. Intenta ejecutar el contenedor en segundo plano (`-d`). Notarás que el contenedor se detiene inmediatamente.
3. Diagnóstico requerido: Identifica el estado del contenedor usando el flag adecuado en el comando de listado e inspecciona los registros para determinar la causa exacta de la parada.



Capturas requeridas para la memoria:

- *Captura 3.1:* Mensaje de error de puerto duplicado en la terminal.
- *Captura 3.2:* Salida de comandos o acceso interactivo que demuestre dónde se guardó la web incorrectamente en el Escenario B.
- *Captura 3.3:* Estado del contenedor detenido (`Exited`) y lectura del error mediante `docker logs` en el Escenario C.

FASE 4: Orquestación con Docker Compose

Reto: Diseñar un infraestructura como código donde pongas en marcha un entorno con dos servidores web Apache simultáneos en una sola instrucción.

1. **Diseño del archivo `docker-compose.yml`:**

- Crea un archivo `docker-compose.yml` en la raíz de tu proyecto.
- Define una versión de sintaxis adecuada.
- Declara dos servicios:
 - **Servicio 1 ("web-personalizada"):** Debe construirse a partir del `Dockerfile` que creaste en la Fase 2 y exponerse en el puerto **8000**.
 - **Servicio 2 ("web-oficial"):** Debe utilizar la imagen directa `httpd:alpine` de Docker Hub y exponerse en el puerto **8001**.
- Define políticas de reinicio (`restart`) para ambos servicios.

2. **Despliegue y Control:**

- Arranca ambos servicios en segundo plano utilizando únicamente un comando de Docker Compose.
- Lista los servicios activos pertenecientes a la orquestación.
- Muestra los registros combinados de ambos servicios en tiempo real.
- Detén y elimina toda la infraestructura orquestada con un solo comando.



Capturas requeridas para la memoria:

- *Captura 4.1:* Contenido completo del archivo `docker-compose.yml`.
- *Captura 4.2:* Salida del terminal al ejecutar la orden de arranque del Compose y la comprobación de estado de los servicios.
- *Captura 4.3:* Ambos sitios web (puerto 8000 y puerto 8001) cargados simultáneamente en pestañas del navegador.